DS	**Session**	Searching	**Question Information**	Level 1 Challenge 3

Problem Description:
Darsh recently found an new rectangular circuit board that he would like to recycle. That has R rows and C columns of squares.

Each square of the circuit board has a very thinness, measured in millimetres. The square in the r-th row and c-th column has thinness Mr,c. A circuit board is beautiful if in each row, the difference between the thinnest square and the least thin square is no greater than L.

Since the original circuit board might not be beautiful, Darsh would like to find a beautiful subcircuit board. This board can be obtained by choosing an axis-aligned subrectangle from the original board and taking the squares in that subrectangle. Darsh would like your help in finding the number of squares in the largest beautiful subrectangle of his original board.

Constraints:
$1 \leq T \leq 50$.
$1 \leq R \leq 205$.
$1 \leq C \leq 205$.
$0 \leq Mi, j \leq 10^3$ for all i, j.
$0 \leq L \leq 10^3$.

Input Format:
The first line of the input gives the number of test cases, T. T test cases follow. Each test case begins with one line containing three integers R, C and L, the number of rows, the number of columns, and the maximum difference in thinness allowed in each row.

Then, there are R more lines containing C integers each. The c-th integer on the r-th line is Mr, c, the thinness of the square in the r-th row and c-th column.

Output Format:
Print the output in a separate lines contains the maximum number of squares in a beautiful subrectangle.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
3
1 4 0
3 1 3 3
2 3 0
4 4 5
7 6 6
4 5 0
2 2 4 4 20
8 3 3 3 12
6 6 3 3 3
1 6 8 6 4
```

EXPECTED OUTPUT

```
2
2
6
```

Test Case 2

INPUT (STDIN)

```
3
2 4 0
2 3 1 2
3 1 3 3
2 3 0
41 4 5
7 61 6
4 5 0
12 12 14 4 20
81 13 13 3 12
61 16 31 3 3
11 16 81 6 4
```

EXPECTED OUTPUT

```
2
2
4
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
int A[309][309];
```

Test Case 2

KEYWORD

```
bool ok[309][309][309];
```

Complexity Test Cases

Test Case 1

Test Case 2

Test Case 3

CYCLOMATIC COMPLEXITY	TOKEN COUNT	NLOC
13	400	53

Code

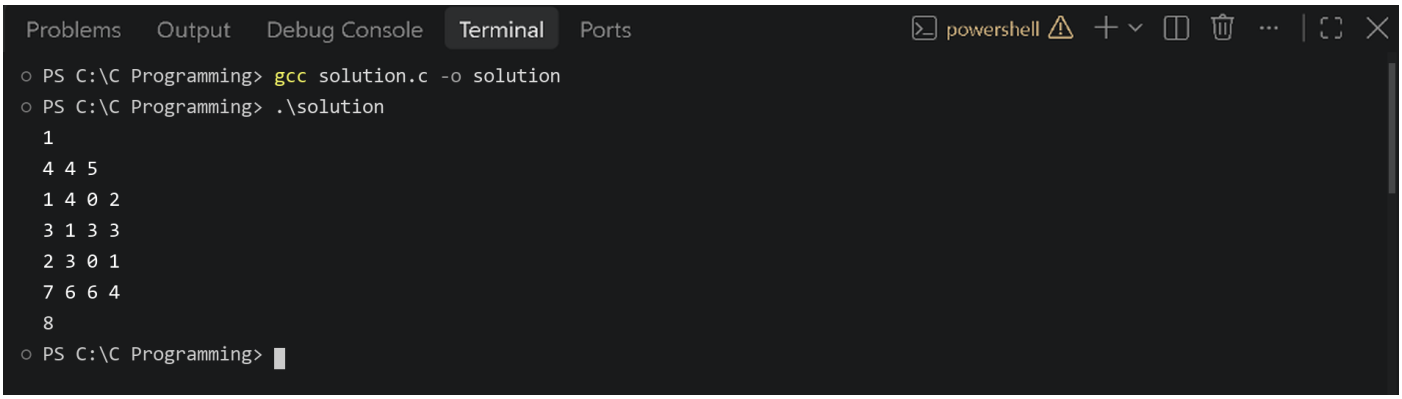
solution.c

```c
#include <stdio.h>

int A[309][309];

int main() {
    int t;
    if (scanf("%d", &t) != 1) return 0;
    while (t--) {
        int r, c, l;
        scanf("%d %d %d", &r, &c, &l);
        for (int i = 0; i < r; i++) {
            for (int j = 0; j < c; j++) {
                scanf("%d", &A[i][j]);
            }
        }
        int maxArea = 0;
        for (int c1 = 0; c1 < c; c1++) {
            for (int c2 = c1; c2 < c; c2++) {
                int curH = 0;
                for (int i = 0; i < r; i++) {
                    int minV = A[i][c1], maxV = A[i][c1];
                    for (int k = c1 + 1; k <= c2; k++) {
                        if (A[i][k] < minV) minV = A[i][k];
                        if (A[i][k] > maxV) maxV = A[i][k];
                    }
                    if (maxV - minV <= l) {
                        curH++;
                        int area = curH * (c2 - c1 + 1);
                        if (area > maxArea) maxArea = area;
                    } else {
                        curH = 0;
                    }
                }
            }
        }
        printf("%d\n", maxArea);
    }
    return 0;
}
```

Output


Problems
Output
Debug Console
Terminal
Ports
powershell
PS C:\C Programming> gcc solution.c -o solution
PS C:\C Programming> .\solution
1
4 4 5
1 4 0 2
3 1 3 3
2 3 0 1
7 6 6 4
8
PS C:\C Programming>